Furnace QR Scanner - Environment Setup

This app is built with the Flutter framework (Dart) and uses Back4App (a hosted Parse Server backed by MongoDB) for its backend. These steps work on Windows, macOS, and Linux.

Prerequisites

- Flutter SDK - Dart SDK >= 3.0.0 < 4.0.0. See https://docs.flutter.dev/get-started/install
- A code editor - VS Code or Android Studio recommended
- An Android emulator or a physical device with USB debugging enabled
- A Back4App account - https://www.back4app.com

Step 1 - Clone the repository

```
git clone <repo-url>
cd furnace-app
```

Step 2 - Verify your Flutter install

Run "flutter doctor" and resolve anything it flags before moving on.

```
flutter doctor
```

Step 3 - Install dependencies

```
flutter pub get
```

Key packages this installs (see pubspec.yaml for the full list):

- parse_server_sdk_flutter - Parse/Back4App client
- mobile_scanner - QR/barcode scanning (replaces the older flutter_barcode_scanner)
- flutter_dotenv - loads Parse credentials from a .env file
- flutter_secure_storage - backs the app's encrypted session-token store
- page_transition, provider, marquee, path_provider, shared_preferences, http

Step 4 - Configure the .env file

Parse credentials are loaded from a .env file at the project root. .env is gitignored and must NOT be committed. An .env.example with the required keys is checked in.

1. Sign in to Back4App and create a new app (or open an existing one).

2. Open App Settings -> Security & Keys and copy the Application ID and Client Key.

3. Copy .env.example to .env at the project root and fill in the values:

```
PARSE_APP_ID=<your application id>
PARSE_CLIENT_KEY=<your client key>
PARSE_SERVER_URL=https://parseapi.back4app.com
```

lib/main.dart calls dotenv.load() on startup and reads PARSE_APP_ID, PARSE_CLIENT_KEY, and PARSE_SERVER_URL. Missing keys will throw at launch.

Step 5 - Create a branch and run the app

```
git checkout -b <your-branch-name>
flutter run
```

In VS Code you can also press F5 to launch the debugger on a running emulator or device.

Backend / Database (Back4App / Parse)

Back4App hosts a Parse Server backed by MongoDB. The app stores users (via ParseUser) and Log entries for room check-ins and check-outs. Standard CRUD patterns with parse_server_sdk_flutter look like:

Create

```
Future<void> saveTodo(String title) async {
  final todo = ParseObject('Todo')
    ..set('title', title)
    ..set('done', false);
  await todo.save();
}
```

Read

```
Future<List<ParseObject>> getTodo() async {
  final query = QueryBuilder<ParseObject>(ParseObject('Todo'));
  final response = await query.query();
  if (response.success && response.results != null) {
    return response.results as List<ParseObject>;
  }
  return [];
}
```

Update

```
Future<void> updateTodo(String id, bool done) async {
  final todo = ParseObject('Todo')
    ..objectId = id
    ..set('done', done);
  await todo.save();
}
```

Delete

```
Future<void> deleteTodo(String id) async {
  final todo = ParseObject('Todo')..objectId = id;
  await todo.delete();
}
```

Session persistence

lib/secure_core_store.dart implements a SecureCoreStore that the Parse SDK uses for session tokens. It is backed by flutter_secure_storage (platform keystore / Keychain) instead of plaintext SharedPreferences, and is wired up in lib/main.dart via Parse().initialize(coreStore: ...).

QR / Barcode Scanning

The app uses the mobile_scanner package. The older flutter_barcode_scanner package referenced in earlier versions of these docs is no longer used - do not add it. Scanner logic lives in lib/scanner.dart.

Package

```
mobile_scanner: ^6.0.0   # already declared in pubspec.yaml
```

Import

```
import 'package:mobile_scanner/mobile_scanner.dart';
```

Typical usage

```
final controller = MobileScannerController();

MobileScanner(
  controller: controller,
  onDetect: (capture) {
    for (final barcode in capture.barcodes) {
      final value = barcode.rawValue; // decoded string
      // ...handle the scan...
    }
  },
);
```

Full docs: https://pub.dev/packages/mobile_scanner

Test QR codes

Pre-generated QR images for local testing live in test-qr/ (for example lab-101-checkin.png, conference-room-checkout.png). The companion shell script scripts/seed-test-rooms.sh seeds matching Room records in your Back4App database. Run it after .env is configured, then scan the PNGs directly from your screen.

Project layout (quick reference)

- lib/main.dart - entry point, Parse init, routing
- lib/login.dart, lib/register.dart, lib/account.dart - auth flows
- lib/welcome.dart - post-login screen
- lib/scanner.dart - QR scanning and log creation
- lib/secure_core_store.dart - encrypted session token storage
- lib/api/ - Parse data access (for example room.dart)
- scripts/seed-test-rooms.sh - seeds test rooms
- test-qr/ - QR fixtures for the seeded rooms